

M14 / STUDY COMPANION

M14 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE14-01 | Batch vs streaming 3

DE14-02 | Events, brokers and topics 3

DE14-03 | Producers and message keys 3

DE14-04 | Consumers and safe processing 3

DE14-05 | Partitions and ordering 3

DE14-06 | Offsets, group position and replay 3

DE14-07 | Consumer groups and scaling 4

DE14-08 | Delivery semantics and idempotency 4

DE14-09 | Serialization, schemas and evolution 4

DE14-10 | Python producer and consumer workflow 4

DE14-11 | Spark Structured Streaming + Kafka 4

DE14-12 | Lag, failures and streaming operations 4

Read this first

Ready-made recall notes. Do not rewrite the learner book. Use this after completing M14 practice.

DE14-01 | Batch vs streaming

Streaming is not chosen because it is fashionable. Choose it when continuous arrival plus latency/replay requirements justify persistent consumer state and operational complexity.

Recall prompt: For three cases - nightly payroll, parcel-location alerting, and a daily regulatory archive - choose batch, streaming, or a hybrid. For each, state the tolerated latency and the replay/recovery expectation.

DE14-02 | Events, brokers and topics

Topics store retained events; brokers host partitions; consumers do not delete records when they read them. Kafka 4.x uses KRaft-era architecture, not ZooKeeper in this course.

Recall prompt: Create three domain topic names for an e-commerce system. For each, write one sentence describing what a single event means and what retention purpose the topic serves.

DE14-03 | Producers and message keys

Use keys to express the unit whose order/grouping matters. Producer durability and delivery confirmation are separate from business-level deduplication downstream.

Recall prompt: Design keys for order-events, customer-profile-changes and device-temperature. State what ordering you need and one hotspot/skew risk for each key.

DE14-04 | Consumers and safe processing

Consumer correctness is the relationship between processing, durable side effects and committed offsets. Prefer replay-safe work over pretending replay never happens.

Recall prompt: Write the safe order of operations for a consumer that writes to PostgreSQL. Include validation, durable write, reconciliation evidence and offset commit.

DE14-05 | Partitions and ordering

Partitions are the unit of parallelism and ordered log storage. Preserve the order that matters by keying related events together, then watch for skew.

Recall prompt: For a food-delivery stream choose a key for driver-location, restaurant-order and payment-event topics. State exactly which order each choice preserves.

DE14-06 | Offsets, group position and replay

Offsets are partition positions; commits are group progress; replay changes reading position while retained log data stays immutable.

Recall prompt: Describe a safe replay plan for rebuilding yesterday's shipping aggregate from retained Kafka events into a new table.

DE14-07 | Consumer groups and scaling

Same group shares partition work; different groups independently consume the same retained stream. Scaling is constrained by partitions and processing cost.

Recall prompt: Given 12 partitions, design consumer counts for email, fraud and analytics applications. Choose group IDs and explain which apps share vs independently read records.

DE14-08 | Delivery semantics and idempotency

Guarantees are scoped. Producer idempotence handles producer retries; replay-safe consumers/sinks handle external effects; Kafka transactions cover Kafka-managed atomic boundaries, not arbitrary external systems.

Recall prompt: For a Kafka -> Python -> PostgreSQL pipeline, state the intended delivery semantics and the exact mechanism preventing duplicate business effects.

DE14-09 | Serialization, schemas and evolution

Kafka moves bytes; schemas create data contracts. Evolve contracts deliberately so producers and consumers can overlap safely during deployment.

Recall prompt: Design v1 and v2 schemas for a parcel event. Make one backward-compatible additive change and one intentionally breaking change. Explain why.

DE14-10 | Python producer and consumer workflow

Python Kafka clients are asynchronous/networked systems. Observe callbacks/errors, control group identity and progress, and make external processing replay-safe.

Recall prompt: Modify the producer to send five order events with order_id keys and modify the consumer group name. Save delivery evidence, consumed partition/offsets and the final ledger count.

DE14-11 | Spark Structured Streaming + Kafka

Kafka retains events; Spark Structured Streaming performs incremental computation and checkpoints its own query state. Treat checkpoints and sink semantics as correctness controls.

Recall prompt: Write a Structured Streaming design for parcel events: Kafka source -> parse -> validate -> aggregate by status -> Delta sink. State the source key, checkpoint location and replay-safe sink assumption.

DE14-12 | Lag, failures and streaming operations

Operate streaming systems per partition and per business freshness objective. Lag is a symptom that needs context, not a diagnosis by itself.

Recall prompt: Write a five-step runbook for rising consumer lag. Include at least one metric, one log/evidence check, one no-data-loss action and one validation after recovery.